



1. (10 نمره)

الف - (5 نمره) تمام ایراد های هر یک از دو برنامه زیر را پیدا کنید و اصلاح شده آنها را بنویسید.

```
#include <stdio.h>
int main() {
    FILE *file;
    file = fopen("output.txt", "w");

    if(file == NULL){
        printf("Could not open file\n");
        return 1;
    }
    fputs("Hello world!\n", file);

    if(fputs("Hello dummy =)\n", file) >= 0){
        printf("Could not open file\n");
    }
    return 0;
}
```

پاسخ:

چک کردن با شرط بزرگتر مساوی صفر غلط است. تابع fputs در صورت موفقیت مقدار غیر منفی و در غیر این صورت EOF را برمیگرداند.

عدم بستن فایل در پایان برنامه: عدم گذاشتن دستور fclose در انتها و قبل از return برای اینکه از buffer overflow جلوگیری شود.

کد اصلاح شده:

```
#include <stdio.h>
int main() {
    FILE *file;
    file = fopen("output.txt", "w");

    if(file == NULL){
        printf("Could not open file\n");
    }
```

```

        return 1;
    }
    fputs("Hello world!\n", file);

    if(fputs("Hello dummy =)\n", file) == EOF){
        printf("Could not open file\n");
    }

    fclose(file);
    return 0;
}

```

ب- (5 نمره) سه ایراد در برنامه زیر را پیدا کنید و اصلاح شده آن را بنویسید.

```

#include <stdio.h>
#include <string.h>

int main() {
    FILE *fp;
    char lines[3][20] = {"Red", "Green", "Blue"};
    char buffer[50];

    fp = fopen("colors.txt", "w");
    if (fp != NULL) {
        for (int i = 0; i < 3; i++) {
            fprintf(fp, "%s\n", lines[i]);
        }
        fclose(fp);
    }
    fp = fopen("colors.txt", "r");

    while (!feof(fp)) {
        fscanf(fp, "%s", buffer);
        printf("Color: %s\n", buffer);
    }
}

```

اشتباه در منطق شرط پایان فایل: استفاده از `while(!feof(fp))` یک اشتباه رایج هست. این تابع فقط پس از برخورد با انتهای فایل فعال می‌شود؛ در نتیجه، حلقه `while`، یک بار بیشتر از حد انتظار اجرا شده و آخرین داده رنگ (Blue) دو بار در خروجی چاپ می‌شود.

عدم مدیریت خطا در گام دوم: پس از باز کردن مجدد فایل در حالت خواندن ("r")، هیچ بررسی برای صحت باز شدن فایل `if(fp != NULL)` انجام نشده است. اگر فایل موجود نباشد، برنامه هنگام خواندن کرش (Crash) می‌کند.

عدم بستن فایل در پایان برنامه: در انتها و قبل از return برای اینکه از buffer overflow جلوگیری شود باید فایل بسته شود (fclose).

کد اصلاح شده :

```
#include <stdio.h>
#include <string.h>

int main() {
    FILE *fp;
    char lines[3][20] = {"Red", "Green", "Blue"};
    char buffer[50];

    fp = fopen("colors.txt", "w");
    if (fp != NULL) {
        for (int i = 0; i < 3; i++) {
            fprintf(fp, "%s\n", lines[i]);
        }
        fclose(fp);
    }

    fp = fopen("colors.txt", "r");
    if (fp == NULL) {
        printf("Could not open file\n");
        return 1;
    }

    //fscanf returns 1 if it successfully reads one item
    while (fscanf(fp, "%s", buffer) == 1) {
        printf("Color: %s\n", buffer);
    }

    fclose(fp);
    return 0;
}
```

2. (5 نمره) خروجی نهایی متن داخل فایل fun.txt را بنویسید (نوشتن متن خروجی بعد از هر دستور fputs الزامی است).

```
#include <stdio.h>

int main() {
    FILE* my_file = fopen("fun.txt", "w");
```

```

fputs("Siasat dar mabani", my_file);
fseek(my_file, 3, SEEK_SET);

fputs("\n", my_file);
fseek(my_file, 9, SEEK_CUR);

fputs("yash", my_file);
fseek(my_file, -12, SEEK_END);

fputs("sh Gho", my_file);
fclose(my_file);

return 0;
}

```

➔ Siasat dar mabani ➔ Siavat dar mabani ➔ Siavat dar mayshi

➔ خروجی نهایی : Siavash Ghomayshi

3. (10 نمره) می‌خواهیم تابعی به نام `filterNumericData` بنویسیم که دو نام فایل را به عنوان ورودی (آرگومان) دریافت کند. این تابع باید فایل اول را که یک فایل متنی (Text) است باز کرده، و فقط اعداد (کاراکترهای '0' تا '9') را از متن استخراج کرده و در فایل دوم بنویسد. سایر کاراکترها (حروف، فواصل و علائم) نباید در فایل خروجی نوشته شوند.

نیازمندی‌های سیستم :

1. تابع باید در صورت باز شدن موفقیت‌آمیز هر دو فایل و اتمام عملیات، مقدار 0 را برگرداند.
2. در صورت بروز خطا در باز کردن هر یک از فایل‌ها، تابع باید مقدار 1 را برگرداند.

```

int filterNumericData(const char *inputFileName, const char *outputFileName){
...
}

```

پاسخ:

```

#include <stdio.h>
int filterNumericData(const char *inputFileName, const char *outputFileName) {

    FILE *src = fopen(inputFileName, "r");
    FILE *dest = fopen(outputFileName, "w");
    int ch;

```

```

if (src == NULL || dest == NULL) {
    if (src != NULL)
        fclose(src);
    return 1;
}
// Read the file character by character until the End Of File (EOF)
while ((ch = fgetc(src)) != EOF) {
    // ASCII range for digits '0' to '9' is 48 to 57
    if (ch >= 48 && ch <= 57) {
        fputc(ch, dest);
    }
}

fclose(src);
fclose(dest);
return 0;
}

```

4. (25 نمره) فایلی با نام students.txt داریم که حاوی اسامی دانشجویان یک کلاس است. نام و نام خانوادگی هر دانشجو (حداکثر 100 کاراکتر) در خطی جدا نوشته شده است. تابع زیر را طوری کامل کنید که نام و نام خانوادگی هر دانشجو را به کاربر نشان داده و وضعیت حضور دانشجو (بصورت یک عدد صحیح 0 یا 1 در ورودی، 0 به معنای غایب بودن و 1 به معنای حاضر بودن) را از کاربر بعنوان ورودی بگیرد. در نهایت این تابع باید نام و نام خانوادگی هر دانشجو را به همراه وضعیت حضور او مانند مثال زیر در فایلی به نام attendance_report.txt بنویسد. توجه کنید که بعضی از خطوط فایل students.txt ممکن است خالی باشند و نباید آنها را در هنگام پردازش در نظر بگیرید.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
void checkAttendance(){
    ...
}

```

(مثال)

students.txt:

Amir Hossein Farhadi → input = 1

Aida MehrAvar → input = 1

Mazyar Chavoshi → input = 0

attendance_report.txt:

Amir Hossein Farhadi: Present

Aida MehrAvar: Present

Mazyar Chavoshi: Absent

راهنمایی : در صورت استفاده از دستور fgets() می توانید از تابع کمکی زیر استفاده کنید:

```
void removeNewline(char *str) {  
...  
}
```

پاسخ :

تابع removeNewLine به این جهت گذاشته شده که دستور fgets وقتی رشته‌ای را دریافت کند، در انتهای آن \n میگذارد که در ادامه اگر \n را حذف نکنیم، در هنگام نوشتن در فایل خروجی، فرمت مدنظر از بین می‌رود. اما اگر از تابع gets استفاده شود این مشکل از بین می‌رود. با این حال از همان تابع fgets استفاده می‌کنیم:

```
#include <stdio.h>  
#define MAX_LEN 100  
void removeNewline(char *str) {  
    while (*str) {  
        if (*str == '\n') {  
            *str = '\0'; //Replace newline with null  
            return;  
        }  
        str++;  
    }  
}  
  
void checkAttendance(){  
  
    FILE *fin = fopen("students.txt", "r");  
    FILE *fout = fopen("attendance_report.txt", "w");  
    char fullName[MAX_LEN];  
    int attendance;  
    if (fin == NULL || fout == NULL) {  
        return 1;  
    }  
}
```

```

while ( fgets(fullName, sizeof(fullName), fin) != NULL ) {

    removeNewline(fullName);
    printf("%s (1: Present / 0: Absent): ", fullName);
    scanf("%d", &attendance);
    getchar(); //Clear the newline character left in the buffer
    if (attendance == 1) {
        fprintf(fout, "%s: Present\n", fullName);
    } else {
        fprintf(fout, "%s: Absent\n", fullName);
    }
}

fclose(fin);
fclose(fout);
}

```

5. (20 نمره) الگوریتم مرتب سازی Selection Sort یکی از الگوریتم های ساده جهت مرتب سازی یک آرایه ای از اعداد است. این الگوریتم بدین صورت است که از عنصر اول آرایه پیمایش میکند و اندیس کوچکترین عنصر در میان اندیس 0 تا n - 1 آرایه را پیدا میکند، سپس با استفاده از این اندیس پیدا شده، محتوای این عنصر را با اولین عنصر آرایه عوض میکند. برای بار دوم از عنصر دوم آرایه پیمایش میکند و اندیس کوچکترین عنصر در میان اندیس 1 تا n - 1 آرایه را پیدا میکند، سپس با استفاده از این اندیس پیدا شده، محتوای این عنصر را با دومین عنصر آرایه عوض میکند. این روند تا آخر آرایه ادامه پیدا میکند تا کل آرایه مرتب شود.

الف- (10 نمره) شبه کد این الگوریتم را بنویسید.

پاسخ:

```

initialize min_index = 0, size = 0
get arr[], size
for i from 0 to size - 1
    min_index = i
    for j from i + 1 to size - 1
        if array[j] < array[min_index] then
            min_index = j

```

```

        end if
    end for

    if min_index ≠ i then
        temp = array[i]
        array[i] = array[min_index]
        array[min_index] = temp
    end if

```

```

end for

```

ب- (10 نمره) تابع آن را در زبان C طوری کامل کنید که آرایه arr و اندازه آن (n) را بگیرد و الگوریتم بالا را روی آن پیاده سازی کند.

```

void selectionSort(int* arr, int n){
    ...
}

```

پاسخ:

```

void selectionSort(int* arr, int n) {
    for(int i = 0; i < n; i++){
        int min_index = i;

        for(int j = i + 1; j < n; j++){
            if(arr[j] < arr[min_index]){
                min_index = j;
            }
        }

        if(min_index != i){
            int temp = arr[i];
            arr[i] = arr[min_index];
            arr[min_index] = temp;
        }
    }
}

```

6. ****امتیازی (10 نمره)** تابع زیر را طوری کامل کنید که نام یک فایل متنی و یک کلمه خاص را از کاربر دریافت کرده، سپس تعداد دفعات تکرار آن کلمه را در طول فایل محاسبه و برگرداند.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int countWordInFile(char *filename, char *targetWord) {
    ...
}
```

دقت کنید که برای مثال اگر کلمه ورودی "hi" باشد، نباید کلماتی نظیر "hi.", "white", "him" شمردن شوند.

پاسخ:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int isPunct(char c) {
    char puncts[] = ".!?:;";
    for (int i = 0; puncts[i] != '\0'; i++) {
        if (c == puncts[i])
            return 1;
    }
    return 0;
}

int countWordInFile(char *filename, char *targetWord) {
    FILE *file = fopen(filename, "r");
    if (file == NULL) {
        printf("Could not open file\n");
        return 0;
    }

    char tempWord[200];
    int count = 0;
    while (fscanf(file, "%199s", tempWord) != EOF) {

        int len = strlen(tempWord);
        int start = 0, end = len - 1;

        // remove punctuations from the beginning
        while (start < len && isPunct(tempWord[start]))
            start++;
    }
}
```

```

    // remove punctuations from the end
    while (end >= 0 && isPunct(tempWord[end]))
        end--;

    if (start > end)
        continue;

    tempWord[end + 1] = '\0';

    if (strcmp(tempWord + start, targetWord) == 0) {
        count++;
    }
}

fclose(file);
return count;
}

```

7. ****امتیازی (20 نمره)** در این سوال شما باید از فایل input.txt که دارای تعدادی سطر دلخواه

است، اطلاعات هر سطر که شامل تعداد مشخص تا عدد طبیعی است را بخوانید، اعداد هر سطر را مرتب کنید و در نهایت، سطر ها را بر اساس مجموع عناصر آنها مرتب کنید. در آخر خروجی را در فایل output.txt چاپ کنید.

فرم اطلاعات داخل فایل input.txt بدین شرح است:

size_1 -> num_1 num_2 num_3 ... num_n
 size_2 -> num_1 num_2 num_3 ... num_m

.

.

مثال فایل input.txt

4 -> 3 1 1 2
 5 -> 6 2 7 1 3
 2 -> 8 8
 4 -> 1 2 1 2
 1 -> 9

1 1 2 2

1 1 2 3

9

8 8

1 2 3 6 7

برای ذخیره اطلاعات سطر ها، میتوانید از توابع تخصیص حافظه پویای malloc و realloc استفاده کنید. حالا با استفاده از تابع selectionSort که در سوال 5 پیاده سازی کردید، باید سطرها را مرتب کنید. به طور کلی تر، این تابع آدرس یک خانه از آرایه را میگیرد و از آن خانه به بعد را به اندازه n خانه مرتب سازی میکند. برای مثال اگر اسم آرایه (آدرس خانه اول آرایه) به همراه طول آرایه به این تابع داده شود، کل آرایه مرتب میشود.

ابتدا باید تابع selectionSort2 را مشابه selectionSort بسازید، به گونه ای که سطر های آرایه پویای دو بعدی را بر اساس ایندکس S مرتب سازی کند (الگوریتم این تابع بسیار شبیه به تابع selectionSort است، با کمی فکر کردن متوجه خواهید شد که نیازی به جا به جایی عنصر به عنصر سطر ها نیست).

```
void selectionSort2(int** arr, int n, int s) {
}
```

حال از این 2 تابع استفاده کرده و مسئله به سادگی قابل حل است.

راهنمایی: هنگام تخصیص حافظه برای ذخیره اعداد هر سطر با استفاده از فضای حافظه heap ، اگر سطری شامل n عدد است، آرایه ای به اندازه $n + 2$ خانه حافظه تخصیص کنید و سعی کنید اطلاعات مفیدی در این 2 حافظه اضافی قرار دهید.

پاسخ: در خانه اول آرایه، اندازه آرایه و در خانه دوم آرایه، مجموع عناصر آرایه را قرار میدهیم. محتوای هر سطر فایل را از خانه سوم به بعد آرایه ذخیره میکنیم. هنگام استفاده از تابع selectionSort باید به جای اسم آرایه (آدرس خانه اول آرایه) از اسم آرایه + 2 (آدرس خانه سوم آرایه) استفاده کنیم که از خانه سوم به بعد آرایه را مرتب سازی کند. همچنین بدیهی است که:

- اندازه آرایه سطر i برابر با arr[i][0] است
- مجموع عناصر آرایه سطر i برابر با arr[i][1] است.

```
#include <stdio.h>
#include <stdlib.h>

void selectionSort(int* arr, int n) {
    for(int i = 0; i < n; i++){
        int min_index = i;

        for(int j = i + 1; j < n; j++){
            if(arr[j] < arr[min_index]){
                min_index = j;
            }
        }

        if(min_index != i){
            int temp = arr[i];
            arr[i] = arr[min_index];
            arr[min_index] = temp;
        }
    }
}

void selectionSort2(int** arr, int n, int s) {
    for(int i = 0; i < n; i++){
        int min_index = i;

        for(int j = i + 1; j < n; j++){
            if(arr[j][s] < arr[min_index][s]){
                min_index = j;
            }
        }

        if(min_index != i){
            int* temp = arr[i];
            arr[i] = arr[min_index];
            arr[min_index] = temp;
        }
    }
}

int main() {
    FILE* fin = fopen("input.txt", "r");
    FILE* fout = fopen("output.txt", "w");
```

```

if(fin == NULL){
    printf("Could not open file\n");
    return 1;
}

int** arrays = NULL;
int rows = 0;

while(1){

    int size, sum = 0;
    if(fscanf(fin, "%d ->", &size) != 1){
        break;
    }

    arrays = (int**)realloc(arrays, sizeof(int*) * (rows + 1));
    int* temp = (int*)malloc(sizeof(int) * (size + 2));

    for(int j = 0; j < size; j++){
        int num;
        fscanf(fin, " %d", &num);
        temp[j+2] = num;
        sum += num;
    }

    temp[0] = size;
    temp[1] = sum;
    arrays[rows] = temp;
    ++rows;
}
fclose(fin);

for(int i = 0; i < rows; i++){
    selectionSort(arrays[i] + 2, arrays[i][0]);
}
selectionSort2(arrays, rows, 1);

for(int i = 0; i < rows; i++){
    for(int j = 0; j < arrays[i][0]; j++){
        fprintf(fout, "%d ", arrays[i][j+2]);
    }
    fprintf(fout, "\n");
}

```

```
fclose(fout);
for (int i = 0; i < rows; i++){
    free(arrays[i]);
}
free(arrays);

return 0;
}
```

توجه کنید که اگر `arr` یک آرایه دو بعدی ایستا باشد، سطرهای آن به صورت پیوسته در حافظه قرار دارند و `arr` هنگام استفاده به یک اشاره گر به آرایه (`int (*arr)[n]`) تبدیل میشود. بنابراین امکان جا به جایی سطرها با تعویض اشاره گرها وجود ندارد؛ اما اگر `arr` یک آرایه دو بعدی پویا باشد، هر سطر میتواند در ناحیه ای مستقل از حافظه قرار بگیرد و `arr` هنگام استفاده به یک اشاره گر به اشاره گر (`int **arr`) تبدیل میشود. بنابراین میتوان با تعویض خود اشاره گرها، سطرهای این آرایه را جا به جا کرد.